Ira A. Fulton Schools of Engineering
Arizona State University

CSE 531: Distributed and Multiprocessor Operating Systems

Client-Centric Consistency Project: Monotonic Writes

Purpose

The goal of this project is to implement the client-centric consistency model on top of the gRPC Project. Your job is to implement the essential functions that enforce the client-centric consistency, specifically Monotonic writes.

Objectives

Learners will be able to:

- Implement the essential functions that enforce the client-centric consistency.
- Enforce the Monotonic writes policy, which extends the implementation of previous interfaces.
- Determine the problem statement.
- Identify the goal of the problem statement.
- List relevant technologies for the setup and their versions.
- Explain the implementation processes.
- Explain implementation results.

Technology Requirements

- Access to GitHub
- Python
- gRPC
- You are required to use the files in the "Projects Overview and Resources" page in the *Welcome and Start Here* module of the course.

Directions

Part 1: Written Report

Your written report must be a single PDF with the correct naming convention: *CSE 531_Your Name_Client-Centric Consistency_Monotonic-Reads_Written Report*.

Using the provided *Learner Template_CSE 531_Your Name_Client-Centric_Consistency_Monotonic-Reads_Written Report*, compose a report addressing the questions:

1. What is the problem statement?
2. What is the goal of the problem statement?
3. What are the relevant technologies for the setup and their versions?
4. What are the implementation processes?
5. What are the implementation results and their justifications?

*Learners may add subheadings on the template to purposefully call attention to specific, organized details.

Part 2: Project Code

Major tasks:

1. Extend the implementation of Branch.Withdraw and Branch.Deposit interface to enforce Ready your Writes policy.

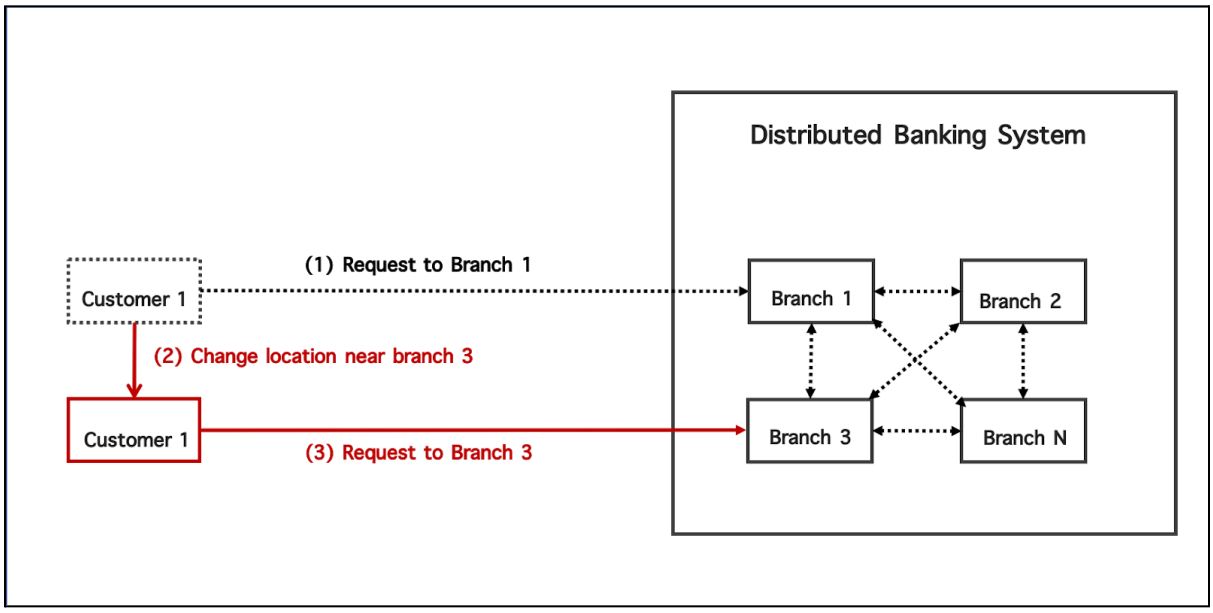


Diagram A: The customer changes the branches while submitting a request to the Bank

1. Description

The customer described in (1) of diagram A, accesses the banking system by connecting to one of the replicas in a transparent way. In other words, the application running on the customer's mobile device is unaware of which replica it is actually using. Assume the customer performs several update operations and then disconnects. Later the customer accesses the banking system again possibly after removing to a different location or using a different access device. At that point the customer may be connected to a different replica than before as shown in (2), (3) of Diagram A. However if the updates performed previously have not yet been propagated, the customer will notice inconsistent behavior. In particular, the customer would expect to see all previously made changes, but instead it appears as if nothing at all has happened. This problem can be alleviated by introducing client-centric consistency. In essence, client-centric consistency provides guarantees for a single client concerning the consistency of accesses to a data store by that client.

1.1. Monotonic Writes

In many situations, it is important that write operations are propagated in the correct order to all copies of the data store. This property is expressed in monotonic-write consistency. In a monotonic-write consistency store, the following condition holds: A write operation by a process on a data item x is completed before any successive write operation on x by the same process. Suppose that a client has an empty bank account and deposits $100 in location A. Suppose that the customer also withdraws while in location B. How and when the deposit of $100 in location A will be transferred to location B is left unspecified. In this case, if the deposit request has not yet been received by the server in location B, then the withdrawal request will fail.

1.2. Implementation

Unlike the gRPC Project, where the customer communicates with only a specific branch with the same unique ID, this project allows the customer to communicate with different branches for query, withdrawal, and deposit operations.

The Branch processes generate the IDs for write operations (deposit and withdraw) requested by the customer. The Customer process obtains the IDs of the write operations performed by the Branch processes. Both the Customer process and the Branch process maintain these sets of write IDs, i.e., writesets. When the Customer sends a request to a Branch, it also sends its writeset. The Branch compares the received writeset to its own writeset, and uses them to enforce client-centric consistency in the banking system.

File structure should remain the same as the gRPC Project. The server and client files should also follow execution as stated in the gRPC Project.

1.2.1. Input and Output

The input file contains one Customer and multiple Branch processes. The format of the input file follows the gRPC Project, you will be using the destination parameter “dest” which has a value of a unique identifier of some branch.

```
[
  {
    "id" : 1,
    "type" : "customer",
    "events" :  [{"interface" : "deposit", "money" : 400, "id" : 1, "dest" : 1},{"interface" : "withdraw", "money" : 400, "id" : 2, "dest" : 2 }, {"interface" : "query", "id" : 3, "dest" : 2 }]
  },
  {
    "id" : 1,
    "type" : "branch",
    "balance" : 0
  },
  {
    "id" : 2,
    "type" : "branch",
    "balance" : 0
  }
]
```

Output file format for the Montonic Writes:

```
[ // array of customers
   { // Customer process #1
     “id” : { a unique identifier of a customer #1  }
      “balance” {the final result of the balance for customer #1}
   }
]
```

1.2.2. Monotonic Writes Example

The test-cases for the Monotonic Write consistency implementation will generate Customer processes that perform write operations on different Branch processes.

Example of the input file

```
[
 {
  "id" : 1,
  "type" : "customer",
  "events" :  [{"interface" : "deposit", "money" : 400, "id" : 1, "dest" : 1},{"interface" : "withdraw", "money" : 400, "id" : 2, "dest" : 2 }, {"interface" : "query", "id" : 3, "dest" : 2 }]
 },
 {
  "id" : 1,
  "type" : "branch",
  "balance" : 0
 },
 {
  "id" : 2,
  "type" : "branch",
  "balance" : 0
 }
]
```

The customer will perform write operations to different Branch processes, and read the final result of the balance from the last Branch process that it requested the write operation. The balance of the customer should reflect the correct number of withdrawal and the deposits.

Expected output file:	**Wrong output file:**
[{"id": 1, "balance": 0 }]	[{"id" : 1, "balance" : 400 }]

Formatting Specifications

Naming convention for your project files and usage with the input file:

- The server file should be named server.py, all required servers should be able to start with this command **python server.py input.json** (execution command to start servers)
- Client file should be named client.py, client file should execute with **python client.py input.json**.

Protobuf: proto file should be inside the protos folder (e.g., **protos/banks.proto**). The expected file structure (excludes files generated after running proto file) should be like this:

```
├──protos
├── ├──banks.proto
├──input.json
├──server.py
├──client.py
├──customer.py
├──branch.py
├──output.json
```

Submission Directions for Project Deliverables

Part 1: Written Report

You are given a limited number of attempts to submit your best work. The number of attempts is given to anticipate any submission errors you may have in regards to properly submitting your best work within the deadline (e.g., accidentally submitting the wrong paper). It is not meant for you to receive multiple rounds of feedback and then one (1) final submission. Only your most recent submission will be assessed.

You must submit your Client-Centric Consistency Project Written Report deliverable in its submission space in the course. Learners may not email or use other means to submit any assignment or project for review, including feedback, and grading.

The Client-Centric Consistency Project Written Report includes one (1) deliverable:

- **Written Report:** Your written report must be a single PDF with the correct naming convention: *CSE 531_Your Name_Client-Centric Consistency_Monotonic-Writes_Written Report.*

Part 2: Project Code

You are given an unlimited number of attempts to submit your best work. You must submit your Client-Centric Consistency Project Code deliverable through Gradescope. Carefully review submission directions outlined in this overview document in order to correctly earn credit for your work. Learners may not email or use other means to submit any assignment or project for review, including feedback, and grading.

Submitting to Gradescope

Your submission will be reviewed by the course team and then, after the due date has passed, your score will be populated from Gradescope into your Canvas grade.

1. Go to the Canvas Assignment, "**Submission: Client-Centric Consistency Project Code**".
2. Click the "**Load Submission…in new window**" button.
3. Once in Gradescope, select the project titled, "**Submission: Client-Centric Consistency-Monotonic-Writes Project Code**", and a pop-up window will appear.
4. In the pop-up,
    a. Submit a single ZIP file.
    b. Click "**Upload**" to submit your work for grading.
5. You will know you have completed the assignment when feedback appears for each test case with a score.
6. If needed: to resubmit the assignment in Gradescope:
    a. Click the "**Resubmit**" button on the bottom right corner of the page and repeat the process from Step 3.

The Client-Centric Consistency Project includes one (1) deliverable:

- **ZIP File:** Your ZIP file must contain your Client-Centric Consistency Project code files and final output. The **code files** must follow the naming conventions outlined in the "Formatting Specifications" section of this Overview Document. The **final output** must be a single JSON file with the correct naming convention: **output.json**.
    - Zip your files by selecting all of them together (your code files should be in the root folder).

Evaluation

Project deliverables will be evaluated based on criteria and will receive a total score.

Evaluation details vary depending on whether the component is auto-graded or course team-graded, so review this section carefully so you understand how you earn credit for each portion of your work.

Review the course syllabus for details regarding late penalties.

Test Case

This component of the project is **auto-graded** and worth **40%** of your project grade.

Your code must be able to handle the following:

- **Monotonic Writes:** The code must be able to perform the Monotonic Writes model and ensure client-centric consistency.

  Example of Monotonic Writes:

  - **Step 1:**
    Customer 1 goes to **Branch 1** and **deposits ₹200**.
    (Balance should now be `200`.)

  - **Step 2:**
    Immediately after, the same customer goes to **Branch 2** and **deposits ₹300**.
    (Balance should now be `500`.)

The system guarantees that the `200` deposit is applied **before** the `300` deposit everywhere.

So the final balance will correctly be `500`

❌ **If Monotonic Writes fails:**

- Branch 2 might process the second deposit (`₹300`) **before** Branch 1's update (`200`).
  - The final balance could end up as `300` instead of `500`.

Your grade for this portion will be assigned based on the **percentage** of returned balance values that are **correct**. This schema for calculating the grade is also provided in the **rubric**:

- About **20%** of the return balance values are correct.
- About **40%** of the returned balance values are correct.
- About **60%** of the returned balance values are correct.
- About **80%** of the returned balance values are correct.
- **All** of the returned balance values are correct.

Code

This component of the project is **course team-graded** and worth **40%** of your project grade.

Review the **rubric** for how your code will be graded.

Project deliverables missing any part of the project will be graded based on what was submitted against the rubric criteria. Missing parts submitted after the deadline will not be graded.

Report

This component of the project is **course team-graded** and worth **20%** of your project grade.

Review the **rubric** for how your written report will be graded.

Project deliverables missing any part of the project will be graded based on what was submitted against the rubric criteria. Missing parts submitted after the deadline will not be graded.

Rubric

Rubrics communicate specific criteria for evaluation. Prior to starting any graded coursework, learners are expected to read through the rubric so they know how they will be assessed. You are encouraged to self-assess your responses and make informed revisions before submitting your final report. Engaging in this learning practice will support you in developing your best work.

Component	Criteria	No Attempt	Undeveloped	Developing	Approaching	Proficient	Exemplary
Test Case	Check if the correct balance is reflected across branches. All transactions should show consistency across all branches.	Provided no response.	About 20% of the returned balance values are correct.	About 40% of the returned balance values are correct.	About 60% of the returned balance values are correct.	About 80% of the returned balance values are correct.	All of the returned balance values are correct.
Component	**Criteria**	**No Attempt**	**Undeveloped**	**Developing**	**Approaching**	**Proficient**	**Exemplary**

Code	The code contains all necessary components, can perform Read Your Writes, and functions correctly.	Provided no response.	Provided project code that is syntactically or semantically invalid.	Provided project code is functional. Project code performs a few of the functions as described in the final report. Project code is not engineered or designed. No documentation is provided.	Provided project code is functional. Project code performs most of the functions as described in the final report. Project code is thought out and engineered. Project code provides documentation and code comments (where appropriate).	Provided project code is functional. Project code performs most of the functions as described in the final report. Project code is thought out and engineered. Project code provides documentation and code comments (where appropriate).	Provided project code that is functional. Project code performs the functions as described in the final report. Project code is excellently engineered. Project code provided helpful documentation and code comments (where appropriate).
Component	**Criteria**	**No Attempt**	**Undeveloped**	**Developing**	**Approaching**	**Proficient**	**Exemplary**
Report	Problem statement and goal	Provided no response.	Provided an incoherent problem statement and goal with no connection to the project description.	Provided a somewhat coherent problem statement and Goal with little or misguided connections with the project description.	Provided a basic, understandable problem statement and goal that loosely connects with the project description.	Provided a clear problem statement and goal that directly connects with the project description.	Provided a focused problem statement and goal that distinctly and in meaningful ways connects with the project description.
	Setup	Provided no response.	Provided an incomplete explanation of the setup.	Provided a reasonable explanation of the setup, but missing steps.	Provided a complete explanation of the setup, but some steps are not working.	Provided a complete and understandable explanation of the setup. All steps are working.	Provided a complete and clear explanation of the setup. All steps are accurate and working correctly.
	Implementation Processes	Provided no response.	Provided an incomplete explanation and/or one or more of the major tasks may be missing.	Provided a general explanation and implementation . Inaccuracies may be present.	Provided a reasonable explanation and implementation . Steps may be illogical or missing.	Provided a logical explanation and implementation . Steps are logical, but may be disjointed or unrelated to one another.	Provided a sound explanation and implementation . All steps are accurate, related, and working correctly.

	Results	Provided no response.	Provided incorrect or fake results. The explanation is fully incorrect with no merit in approach or connection with the results.	Provides some results and/or unrelated results. The explanation is somewhat incorrect with little merit in approach or connection with the results.	Provides mostly correct results. The explanation is reasonable with some ambiguity.	Results are accurate. The explanation is logical with no ambiguity.	Results are accurate. The explanation is well-developed with strong justifications directly related to the implementation results.